SOUTHEAST UNIVERSITY

Meeting the Challenges of Time

Department Of CSE

OS Lab Report

Report No:01

Date of submission: 1 June 2026

Course name : CSE LAB

Course code : CSE362

Section : 6

Submitted To : Shujoy Kundu

[lecturer, Department of CSE]

Initial : **SHKU**

Student's Name : Nazmul Hasan

Student's ID : 2023100000130

Objectives:

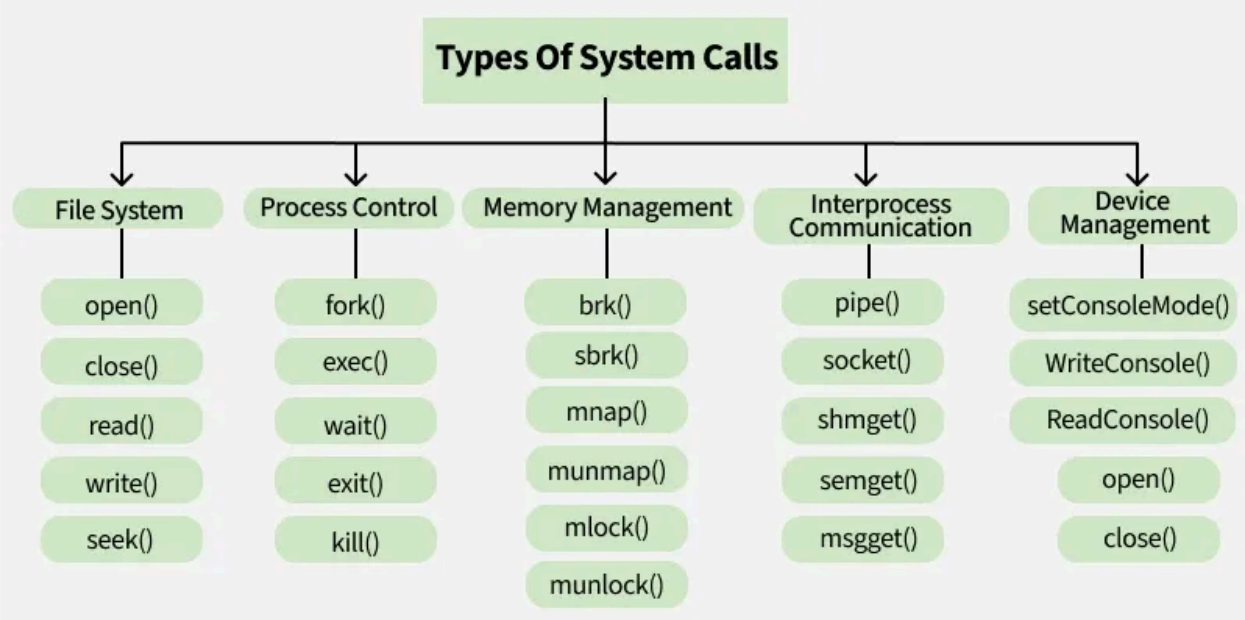
- To understand the fundamental concept of system calls and the role of process management in Unix/Linux operating systems.
- To practically implement and analyze the working mechanism of the fork() system call, specifically focusing on process creation, return values, and parent-child execution behavior.

Introduction:

A system call is a programmatic method by which a computer program requests a service from the kernel of the operating system. Because user applications run in a restricted "user mode," they cannot directly access hardware, memory, or CPU resources. System calls act as the bridge allowing these operations safely**.**

Process management is one of the most critical functions of an operating system. A process is simply a program in execution. The OS is responsible for creating, scheduling, and terminating these processes. Without proper process management, multiple programs running simultaneously would interfere with one another, leading to crashes and inefficient CPU utilization.

Types of System Calls



Theory/Methodology:

A **System Call** is the programmatic way a user-space application requests a service from the operating system's kernel. Programs typically request these services through a high-level Application Programming Interface (API), which acts as a safe bridge between the user and the OS.

To maintain system stability and security, operating systems utilize two distinct execution modes:

- **User Mode:** The restricted environment where standard user code executes without direct access to hardware or memory.
- **Kernel Mode:** The privileged environment where the OS operates with unrestricted access to all system resources.

When a user program needs a resource (such as creating a new process or reading a file), it makes a system call. This triggers a **Trap** (a software interrupt), which safely switches the CPU from User Mode to Kernel Mode to perform the necessary task before switching back.

The fork() System Call

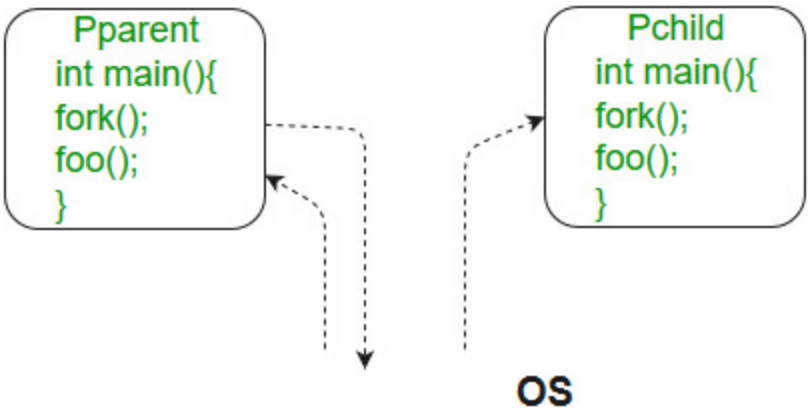
The fork system call is used for creating a new process in Linux, and Unix systems, which is called the ***child process***, which runs concurrently with the process that makes the fork() call (parent process). After a new child process is created, both processes will execute the next instruction following the fork() system call.

The child process uses the same pc(program counter), same CPU registers, and same open files which are used in the parent process. It takes no parameters and returns an integer value.

Below are different values returned by **fork().**

- **Negative Value**: The creation of a child process was unsuccessful, then it returns -1.
- **Zero**: Returned to the newly created child process.
- **Positive value**: Returned to parent or caller. The value contains the process ID of the newly created child process.

Behavior: The parent and child execute concurrently. The parent can either continue executing simultaneously with the child, or it can use the wait() system call to pause until the child terminates.



Basic Terminologies Used in Fork System Call in Operating System

Process: In an operating system, a process is an instance of a program that is currently running. It is a separate entity with its own memory, resources, CPU, I/O hardware, and files.

Parent Process: The process that uses the fork system call to start a new child process is referred to as the parent process. It acts as the parent process's beginning point and can go on running after the fork.

Child Process: The newly generated process as a consequence of the fork system call is referred to as the child process. It has its own distinct process ID (PID), and memory, and is a duplicate of the parent process.

Process ID: A process ID (PID) is a special identification that the operating system assigns to each process.

Copy-on-Write: The fork system call makes use of the memory management strategy known as copy-on-write. Until one of them makes changes to the shared memory, it enables the parent and child processes to share the same physical memory. To preserve data integrity, a second copy is then made.

Return Value: The fork system call's return value gives both the parent and child process information. It assists in handling mistakes during process formation and determining the execution route.

Overview of the use and significance of the fork() system call:

Process Creation: The primary purpose of the **fork()** system call is to create a new process, often referred to as the child process. After the **fork()** call, two processes exist: the parent process and the child process. Both processes have identical copies of the memory, file descriptors, and other resources of the parent process at the time of the **fork()** call.

Parallel Execution: By creating a child process, the **fork()** system call enables parallel code execution. Each process (parent and child) runs independently and can perform different tasks simultaneously.

Process Control: The **fork()** system call is fundamental for process control in Unix-like operating systems. It allows programs to spawn new processes, which can execute different sections of code or perform different tasks concurrently.

Multiple Processes: The ability to create multiple processes through **fork()** facilitates multitasking and multitasking environments. It allows programs to perform several tasks simultaneously, such as handling multiple client connections in a server application or executing multiple computations in parallel.

Process Hierarchy: In Unix-like operating systems, processes are organized hierarchically. The process that calls **fork()** becomes the parent process, and the newly created process becomes its child. This hierarchy enables the management and coordination of processes in the system.

Program Code And Execution:

Example 1: Basic Process Segregation

Code:

```c
#include <stdio.h>
#include <unistd.h>
int main() {
    pid_t pid = fork(); // Create a new process
    if (pid == 0) {
        // This is the child process
        printf("Hello from the child process!\n");
    } else {
        // This is the parent process
        printf("Hello from the parent process!\n");
    }
    return 0;
}
```

Output:

```
nazmul@DESKTOP-09D7DST:~/desktop/cse362.6/documents/labreport$ vim fork1.c
nazmul@DESKTOP-09D7DST:~/desktop/cse362.6/documents/labreport$ gcc fork1.c -o fork1
nazmul@DESKTOP-09D7DST:~/desktop/cse362.6/documents/labreport$ ./fork1\
> ^C
nazmul@DESKTOP-09D7DST:~/desktop/cse362.6/documents/labreport$ ./fork1
Hello from the parent process!
Hello from the child process!
```

Explanation:

- The fork() call duplicates the process; the OS assigns a return value of 0 to the child and a positive PID to the parent.
- The if-else condition acts as a router, forcing the child and parent to execute completely different blocks of code.
- Because both processes run concurrently, the output order of these two printf statements is determined by the OS CPU scheduler.

Example 2: Multiple Process Creation

Code:

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
//main function begins
int main(){
    fork();
    fork();
    fork();
    printf("this process is created by fork() system call\n");
    return 0;
}
```

Output:

```
nazmul@DESKTOP-09D7DST:~/desktop/cse362.6/documents/labreport$ vim fork1.c
nazmul@DESKTOP-09D7DST:~/desktop/cse362.6/documents/labreport$ vim fork2.c
nazmul@DESKTOP-09D7DST:~/desktop/cse362.6/documents/labreport$ gcc fork2.c -o fork2
nazmul@DESKTOP-09D7DST:~/desktop/cse362.6/documents/labreport$ ./fork2
this process is created by fork() system call
this process is created by fork() system call
this process is created by fork() system call
this process is created by fork() system call
this process is created by fork() system call
this process is created by fork() system call
nazmul@DESKTOP-09D7DST:~/desktop/cse362.6/documents/labreport$ this process is created by fork() system call
this process is created by fork() system call
```

Example 3: Process Replacement and Synchronization

Code:

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    // pid_t is the datatype used to store process IDs.
    pid_t pid = fork();

    if (pid < 0) {
        // fork failed
        printf("fork failed!\n");
    }
    else if (pid == 0) {
        // Child process
        printf("Child: running 'ls -l'\n");
        execlp("ls", "ls", "-l", NULL);
        printf("Child: exec failed\n"); // Runs only if exec fails
    }
    else {
        // Parent process
        printf("Parent: waiting for child...\n");
        wait(NULL);
        printf("Parent: child finished.\n");
    }

    return 0;
}
```

Output:

```
nazmul@DESKTOP-09D7DST:~/desktop/cse362.6/documents/labreport$ gcc fork.c -o fork
nazmul@DESKTOP-09D7DST:~/desktop/cse362.6/documents/labreport$ ./fork
Parent: waiting for child...
Child: running 'ls -l'
total 20
-rwxr-xr-x 1 nazmul nazmul 16088 Jun  1 01:42 fork
-rwxr-xr-x 1 nazmul nazmul   621 Jun  1 01:41 fork.c
Parent: child finished.
```

Explanation:

- The child process uses the `execlp()` system call to completely replace its duplicated code with a new program (the Linux `ls -l` command).
- The parent process uses the `wait(NULL)` system call to pause its own execution until the child process finishes its task.

- This synchronization prevents the child from becoming a "zombie" process and ensures the parent only resumes after the child terminates safely.

Observations and Process Creation Behavior

- **a) Concurrency and Scheduling Indeterminism:** One of the most critical observations when running code containing fork() is that the exact order of the output is not guaranteed. Once the child process is created, both the parent and child processes compete for CPU time. Depending on the operating system's CPU scheduler (like the Completely Fair Scheduler in Linux) and current system load, the child process might finish execution and print its output before the parent even begins its next line of code, or vice versa .
- **b) The Multiplying Effect (2^n):** As demonstrated in sequential fork() execution blocks, calling fork() back-to-back without conditional boundaries creates an exponential growth tree. A process structure running n consecutive forks will result in 2^n total active execution processes because both the parent and all newly created children continue executing the subsequent fork() commands.
- **c) Copy-On-Write (CoW) Mechanism:** Although the child is an exact duplicate of the parent, they do not immediately consume double the physical memory. Modern Unix/Linux systems use a memory management strategy called Copy-on-Write. The parent and child share the same physical memory pages until one of them attempts to modify a variable. Only at the moment of modification does the OS create a separate, private copy of that specific memory page to ensure process isolation

Advantages of Fork System Call

Creating new processes with the fork system call facilitates the running of several tasks concurrently within an operating system. The system's efficiency and multitasking skills are improved by this concurrency.

Code reuse: The child process inherits an exact duplicate of the parent process, including every code segment, when the fork system call is used. By using existing code, this feature encourages code reuse and streamlines the creation of complicated programmes.

Memory Optimisation: When using the fork system call, the copy-on-write method optimises the use of memory. Initial memory overhead is minimised since parent and child processes share the same physical memory pages. Only when a process changes a shared memory page, improving memory efficiency, does copying take place.

Process isolation is achieved by giving each process started by the fork system call its own memory area and set of resources. System stability and security are improved because of this isolation, which prevents processes from interfering with one another.

Disadvantages of Fork System Call

Memory Overhead: The fork system call has memory overhead even with the copy-on-write optimisation. The parent process is first copied in its entirety, including all of its memory, which increases memory use.

Duplication of Resources: When a process forks, the child process duplicates all open file descriptors, network connections, and other resources. This duplication may waste resources and perhaps result in inefficiencies.

Communication complexity: The fork system call generates independent processes that can need to coordinate and communicate with one another. To enable data transmission across processes, interprocess communication methods, such as pipes or shared memory, must be built, which might add complexity.

Impact on System Performance: Forking a process duplicates memory allocation, resource management, and other system tasks. Performance of the system may be affected by this, particularly in situations where processes are often started and stopped.

Conclusion

In conclusion, the fork system call in operating systems offers a potent mechanism for process formation. It enables the execution of numerous tasks concurrently and the effective use of system resources by replicating the parent process. It's crucial for the creation of reliable and effective operating systems to comprehend the complexities of the fork system call.

References:

1. https://www.geeksforgeeks.org/operating-systems/introduction-of-system-call/
2. https://www.geeksforgeeks.org/c/fork-system-call/
3. https://www.geeksforgeeks.org/operating-systems/fork-system-call-in-operating-system/
4. https://medium.com/@namobilegame/operating-sytem-fork-system-call-0b85c0dc96be
5. https://drive.google.com/file/d/153xQdeBlUudIZLhvszFFAKJ8qhbflFFT/view fork.pdf
6. https://os.ecci.ucr.ac.cr/slides/Abraham-Silberschatz-Operating-System-Concepts-10th-2018.pdf